House Price Prediction Model

A machine learning project that predicts residential property prices.

The Problem

Predicting house prices is difficult — value depends on dozens of interacting factors like size, location, age, and number of rooms, and the relationships are rarely linear. This project builds a data-driven regression model that learns patterns from real property data to estimate sale prices.

The project runs across two tracks:

- Main project — Melbourne property prices using melb_data.csv (13,580 listings, target: Price in AUD)
- Kaggle exercises — Ames, Iowa home prices using the [Kaggle Housing Prices Competition] dataset (79 features, ~1,460 training samples)

Why It Matters

Housing is one of the largest financial decisions most people make. A reliable prediction model gives buyers a benchmark for fair pricing, helps sellers set competitive prices, and gives lenders a consistent way to assess collateral — without waiting for a manual appraisal. From a learning standpoint, this problem covers nearly every fundamental ML skill: cleaning, encoding, imputing, training, tuning, and evaluating.

Approach

Main Project Pipeline:

1. Load and explore melb_data.csv — 13,580 rows, 21 columns, price range $85K–$9M
2. Drop rows with missing values in numeric features → 6,830 clean rows remain
3. Auto-select 12 numeric features: Rooms, Distance, BuildingArea, YearBuilt, Landsize, Bathroom, Car, Postcode, Bed room2, Lattitude, Longtitude, Propertycount
4. 80/20 train-validation split (random_state=1)
5. Train Decision Tree (baseline + tuned) and Random Forest (100 trees + cross-validated)
6. Evaluate on MAE, RMSE, and R^2

Kaggle Exercises — what each one covered:

Exercise	Topic
Introduction	Comparing 5 Random Forest variants, selecting best

Exercise	Topic
Missing Values	Drop columns vs. mean imputation
Categorical Variables	Drop vs. ordinal encoding vs. one-hot encoding
Pipelines	Bundling preprocessing + model into sklearn.Pipeline
XGBoost	Gradient boosting, tuning n_estimators and learning_rate
Data Leakage	Identifying target leakage and train-test contamination

---

Key Decisions

- **Dropped rows over imputing — BuildingArea was the most important feature (37% of RF importance), so filling 6,450 missing values with a mean would have corrupted the model's primary signal**
- **Ordinal encoding beat one-hot — Empirically produced the lowest MAE (17,098) vs. dropping (17,837) or one-hot (17,525) on the Ames dataset**
- **n_estimators = 150 — Cross-validation showed no meaningful gain beyond 150 trees; MAE plateaued at $211,795**
- **Mean imputation over constant — More statistically appropriate, reduced MAE from 17,614 → 17,612 in the pipeline exercise**

Results

Melbourne Housing

Model	MAE
Decision Tree (baseline)	$229,232
Decision Tree (tuned, max_leaf_nodes=500)	$208,956
Random Forest (100 trees)	$173,738

R^2 Score: 0.7937 — Mean Percentage Error: 16.14%

Top features: BuildingArea (0.375) › Distance (0.156) › Postcode (0.110) › YearBuilt (0.101)

Kaggle — Ames, Iowa

Approach	MAE
Ordinal encoding + Random Forest	17,098
Pipeline (mean imputation + one-hot + RF)	17,612
XGBoost (tuned)	17,032

What I Learned

- Data preparation matters more than algorithm choice — cleaning alone took more time than modelling
- Feature importances matched real-world logic: size and location dominate Melbourne prices
- Empirical results beat assumptions — imputation didn't beat dropping, and ordinal beat one-hot, contrary to expectation
- Pipelines are essential for consistent, deployment-safe preprocessing
- Data leakage can make a model look perfect in training while being useless in production

---

Tech Stack

Python · Pandas · NumPy · Scikit-learn · XGBoost · Matplotlib · Jupyter Notebook

---

Project Structure

```
House_price_prediction_model/
 ├── Real_Estate_Price_Prediction_model.ipynb
 ├── melb_data.csv
 ├── exercise-introduction.ipynb
 ├── exercise-missing-values.ipynb
 ├── exercise-categorical-variables.ipynb
 ├── exercise-pipelines.ipynb
 ├── exercise-xgboost.ipynb
 ├── exercise-data-leakage.ipynb
 └── README.md
```

Getting Started

git clone https://github.com/khanmdraza2029-dev/House_price_prediction_model.git

cd House_price_prediction_model

pip install pandas numpy scikit-learn xgboost matplotlib jupyter

jupyter notebook Real_Estate_Price_Prediction_model.ipynb

Kaggle exercise notebooks use ../input/ paths and the learntools library — run them inside the Kaggle notebook environment.

House Price Prediction Model

A machine learning project that predicts residential property prices.

The Problem

Predicting house prices is difficult — value depends on dozens of interacting factors like size, location, age, and number of rooms, and the relationships are rarely linear. This project builds a data-driven regression model that learns patterns from real property data to estimate sale prices.

The project runs across two tracks:

- **Main project** — Melbourne property prices using melb_data.csv (13,580 listings, target: Price in AUD)
- **Kaggle exercises** — Ames, Iowa home prices using the Kaggle Housing Prices Competition dataset (79 features, ~1,460 training samples)

Why It Matters

Housing is one of the largest financial decisions most people make. A reliable prediction model gives buyers a benchmark for fair pricing, helps sellers set competitive prices, and gives lenders a consistent way to assess collateral — without waiting for a manual appraisal. From a learning standpoint, this problem covers nearly every fundamental ML skill: cleaning, encoding, imputing, training, tuning, and evaluating.

Approach

Main Project Pipeline:

1. Load and explore melb_data.csv — 13,580 rows, 21 columns, price range $85K–$9M
2. Drop rows with missing values in numeric features → 6,830 clean rows remain
3. Auto-select 12 numeric features: Rooms, Distance, BuildingArea, YearBuilt, Landsize, Bathroom, Car, Postcode, Bedroom2, Lattitude, Longtitude, Propertycount
4. 80/20 train-validation split (random_state=1)

5. Train Decision Tree (baseline + tuned) and Random Forest (100 trees + cross-validated)
6. Evaluate on MAE, RMSE, and R^2

Kaggle Exercises — what each one covered:

Exercise	Topic
Introduction	Comparing 5 Random Forest variants, selecting best
Missing Values	Drop columns vs. mean imputation
Categorical Variables	Drop vs. ordinal encoding vs. one-hot encoding
Pipelines	Bundling preprocessing + model into sklearn.Pipeline
XGBoost	Gradient boosting, tuning n_estimators and learning_rate
Data Leakage	Identifying target leakage and train-test contamination

---

Key Decisions

- **Dropped rows over imputing** — BuildingArea was the most important feature (37% of RF importance), so filling 6,450 missing values with a mean would have corrupted the model's primary signal
- **Ordinal encoding beat one-hot** — Empirically produced the lowest MAE (17,098) vs. dropping (17,837) or one-hot (17,525) on the Ames dataset
- **n_estimators = 150** — Cross-validation showed no meaningful gain beyond 150 trees; MAE plateaued at $211,795
- **Mean imputation over constant** — More statistically appropriate, reduced MAE from 17,614 → 17,612 in the pipeline exercise

---

Results

Melbourne Housing

Model	MAE
Decision Tree (baseline)	$229,232
Decision Tree (tuned, max_leaf_nodes=500)	$208,956
Random Forest (100 trees)	**$173,738**

R^2 Score: **0.7937** — Mean Percentage Error: **16.14%**

Top features: BuildingArea (0.375) › Distance (0.156) › Postcode (0.110) › YearBuilt (0.101)

Kaggle — Ames, Iowa

Approach	MAE
Ordinal encoding + Random Forest	17,098
Pipeline (mean imputation + one-hot + RF)	17,612
XGBoost (tuned)	**17,032**

What I Learned

- Data preparation matters more than algorithm choice — cleaning alone took more time than modelling
- Feature importances matched real-world logic: size and location dominate Melbourne prices
- Empirical results beat assumptions — imputation didn't beat dropping, and ordinal beat one-hot, contrary to expectation
- Pipelines are essential for consistent, deployment-safe preprocessing
- Data leakage can make a model look perfect in training while being useless in production

---

Tech Stack

Python · Pandas · NumPy · Scikit-learn · XGBoost · Matplotlib · Jupyter Notebook

---

Project Structure

House_price_prediction_model/

```
├── Real_Estate_Price_Prediction_model.ipynb
├── melb_data.csv
├── exercise-introduction.ipynb
├── exercise-missing-values.ipynb
├── exercise-categorical-variables.ipynb
├── exercise-pipelines.ipynb
├── exercise-xgboost.ipynb
├── exercise-data-leakage.ipynb
```

└── README.md

Getting Started

```
git clone https://github.com/khanmdraza2029-dev/House_price_prediction_model.git

cd House_price_prediction_model

pip install pandas numpy scikit-learn xgboost matplotlib jupyter

jupyter notebook Real_Estate_Price_Prediction_model.ipynb
```

Kaggle exercise notebooks use ../input/ paths and the learntools library — run them inside the Kaggle notebook environment.